PRGRM1

February 11, 2026

[1]:
```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
```

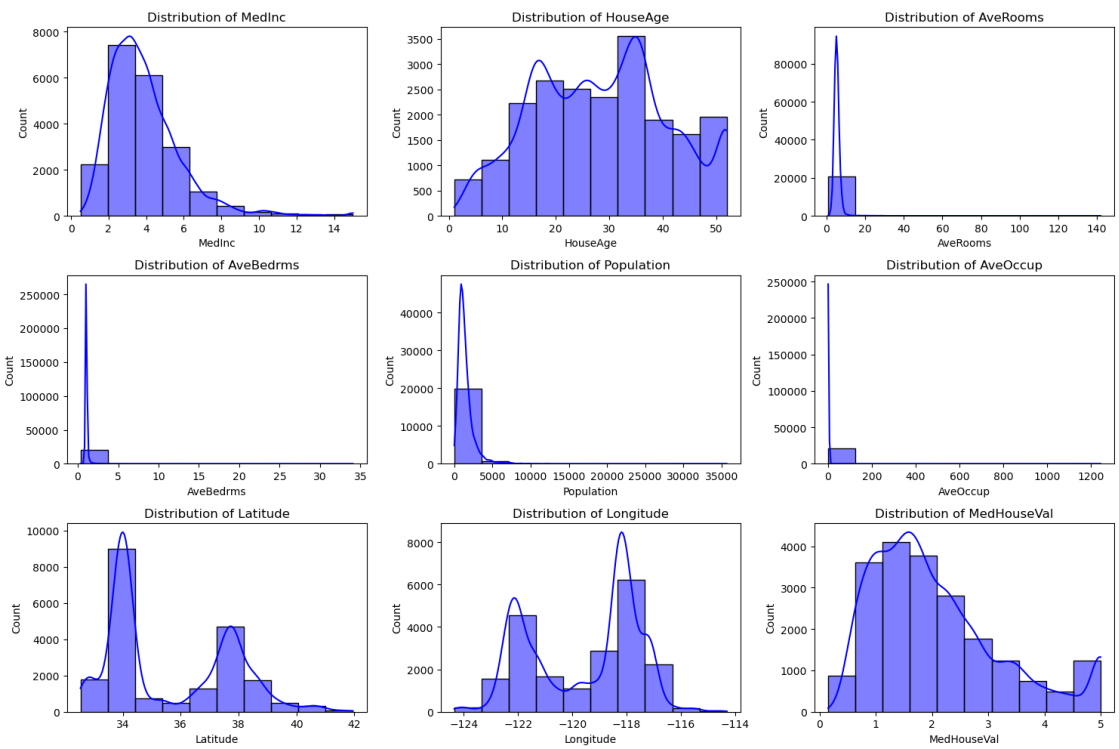
[2]:
```python
data = fetch_california_housing(as_frame=True)
housing_df = data.frame
print(housing_df)
```

```
       MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0      8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1      8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2      7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3      5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4      3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85
…       …       …        …         …          …        …        …
20635  1.5603      25.0  5.045455   1.133333       845.0  2.560606     39.48
20636  2.5568      18.0  6.114035   1.315789       356.0  3.122807     39.49
20637  1.7000      17.0  5.205543   1.120092      1007.0  2.325635     39.43
20638  1.8672      18.0  5.329513   1.171920       741.0  2.123209     39.43
20639  2.3886      16.0  5.254717   1.162264      1387.0  2.616981     39.37

       Longitude  MedHouseVal
0        -122.23        4.526
1        -122.22        3.585
2        -122.24        3.521
3        -122.25        3.413
4        -122.25        3.422
…          …           …
20635    -121.09        0.781
20636    -121.21        0.771
20637    -121.22        0.923
20638    -121.32        0.847
20639    -121.24        0.894

[20640 rows x 9 columns]
```

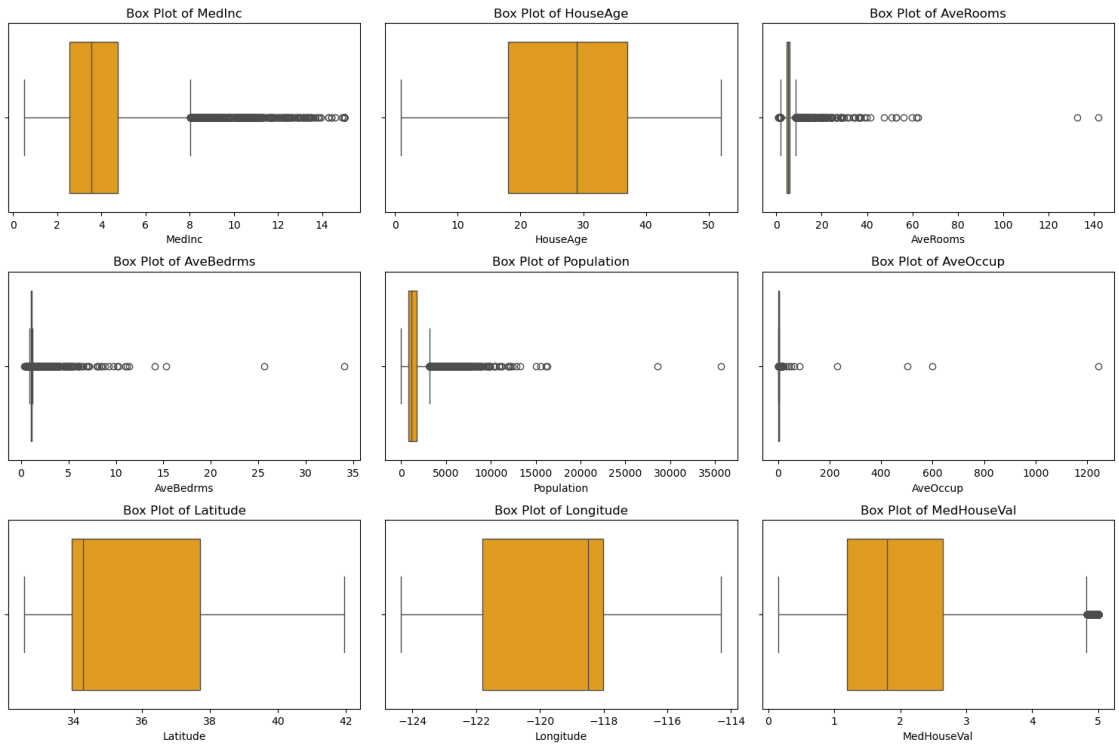
[3]:
```
numerical_features = housing_df.select_dtypes(include=[np.number]).columns
print(numerical_features)
```

```
Index(['MedInc', 'HouseAge', 'AveRooms', 'AveBedrms', 'Population', 'AveOccup',
       'Latitude', 'Longitude', 'MedHouseVal'],
      dtype='object')
```

[5]:
```
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3,3,i+1)
    sns.histplot(housing_df[feature],kde=True,bins=10,color='blue')
    plt.title(f'Distribution of {feature}')
plt.tight_layout()
plt.show()
```



[7]:
```
plt.figure(figsize=(15, 10))
for i, feature in enumerate(numerical_features):
    plt.subplot(3,3,i+1)
    sns.boxplot(x=housing_df[feature],color='orange')
    plt.title(f'Box Plot of {feature}')
plt.tight_layout()
plt.show()
```



[14]:
```
print("Outlier Detection:")
outlier_summary = {}
for feature in numerical_features:
    Q1 = housing_df[feature].quantile(0.25)
    Q3 = housing_df[feature].quantile(0.75)
    IQR = Q3 - Q1
    lower_bound = Q1 - 1.5 * IQR
    upper_bound = Q3 + 1.5 * IQR
    outlier = housing_df[(housing_df[feature] < lower_bound) |␣
 ↪(housing_df[feature] > upper_bound)]
    outlier_summary[feature] = len(outlier)
    print(f"{feature}: {len(outlier)} outliers")
```

```
Outlier Detection:
MedInc: 681 outliers
HouseAge: 0 outliers
AveRooms: 511 outliers
AveBedrms: 1424 outliers
Population: 1196 outliers
AveOccup: 711 outliers
Latitude: 0 outliers
Longitude: 0 outliers
MedHouseVal: 1071 outliers
```

[15]:
```python
print("\nDataset Summary:")
print(housing_df.describe())
```

```
Dataset Summary:
             MedInc      HouseAge      AveRooms     AveBedrms    Population  \
count  20640.000000  20640.000000  20640.000000  20640.000000  20640.000000
mean       3.870671     28.639486      5.429000      1.096675   1425.476744
std        1.899822     12.585558      2.474173      0.473911   1132.462122
min        0.499900      1.000000      0.846154      0.333333      3.000000
25%        2.563400     18.000000      4.440716      1.006079    787.000000
50%        3.534800     29.000000      5.229129      1.048780   1166.000000
75%        4.743250     37.000000      6.052381      1.099526   1725.000000
max       15.000100     52.000000    141.909091     34.066667  35682.000000

           AveOccup      Latitude     Longitude   MedHouseVal
count  20640.000000  20640.000000  20640.000000  20640.000000
mean       3.070655     35.631861   -119.569704      2.068558
std       10.386050      2.135952      2.003532      1.153956
min        0.692308     32.540000   -124.350000      0.149990
25%        2.429741     33.930000   -121.800000      1.196000
50%        2.818116     34.260000   -118.490000      1.797000
75%        3.282261     37.710000   -118.010000      2.647250
max     1243.333333     41.950000   -114.310000      5.000010
```

[ ]:
```python

```

PRGRM2

February 11, 2026

[1]:
```
import pandas as pd
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.datasets import fetch_california_housing
```

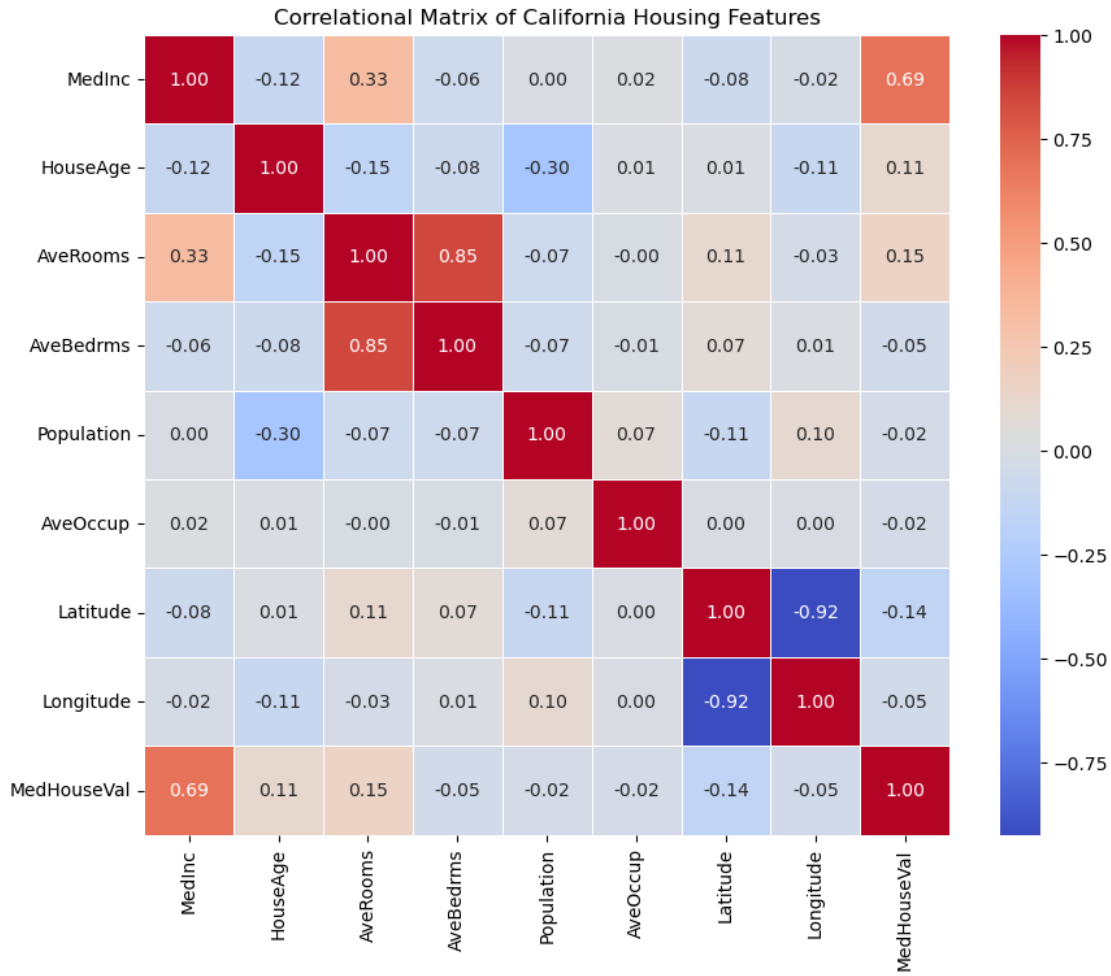
[2]:
```
california_data = fetch_california_housing(as_frame=True)
data = california_data.frame
print(data)
```

```
       MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  Latitude  \
0      8.3252      41.0  6.984127   1.023810       322.0  2.555556     37.88
1      8.3014      21.0  6.238137   0.971880      2401.0  2.109842     37.86
2      7.2574      52.0  8.288136   1.073446       496.0  2.802260     37.85
3      5.6431      52.0  5.817352   1.073059       558.0  2.547945     37.85
4      3.8462      52.0  6.281853   1.081081       565.0  2.181467     37.85
…       …       …        …         …          …        …       …
20635  1.5603      25.0  5.045455   1.133333       845.0  2.560606     39.48
20636  2.5568      18.0  6.114035   1.315789       356.0  3.122807     39.49
20637  1.7000      17.0  5.205543   1.120092      1007.0  2.325635     39.43
20638  1.8672      18.0  5.329513   1.171920       741.0  2.123209     39.43
20639  2.3886      16.0  5.254717   1.162264      1387.0  2.616981     39.37

       Longitude  MedHouseVal
0        -122.23        4.526
1        -122.22        3.585
2        -122.24        3.521
3        -122.25        3.413
4        -122.25        3.422
…          …          …
20635    -121.09        0.781
20636    -121.21        0.771
20637    -121.22        0.923
20638    -121.32        0.847
20639    -121.24        0.894

[20640 rows x 9 columns]
```

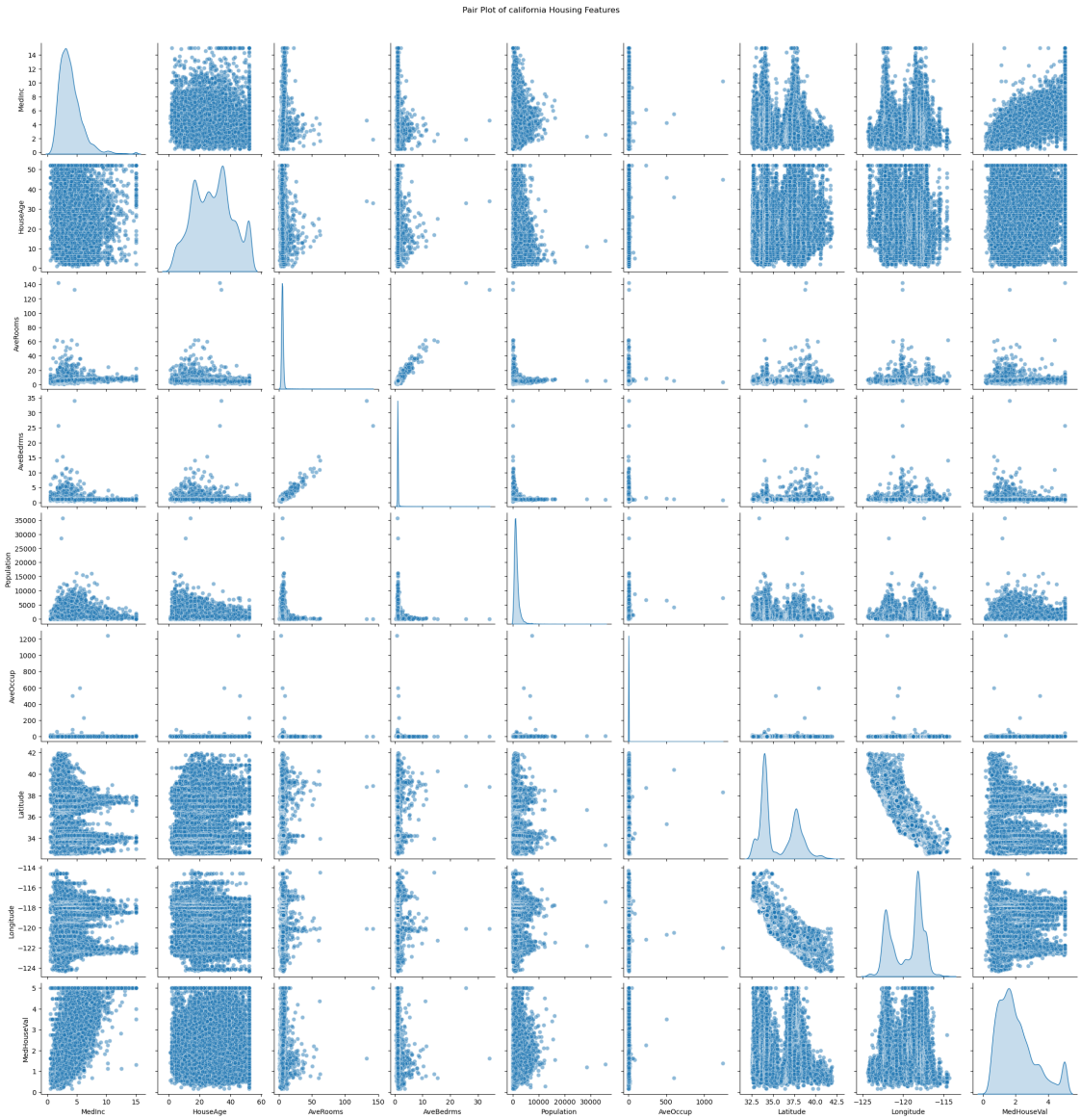
[3]:
```
correlation_matrix = data.corr()
print(correlation_matrix)
```

```
               MedInc  HouseAge  AveRooms  AveBedrms  Population  AveOccup  \
MedInc       1.000000 -0.119034  0.326895  -0.062040    0.004834  0.018766
HouseAge    -0.119034  1.000000 -0.153277  -0.077747   -0.296244  0.013191
AveRooms     0.326895 -0.153277  1.000000   0.847621   -0.072213 -0.004852
AveBedrms   -0.062040 -0.077747  0.847621   1.000000   -0.066197 -0.006181
Population   0.004834 -0.296244 -0.072213  -0.066197    1.000000  0.069863
AveOccup     0.018766  0.013191 -0.004852  -0.006181    0.069863  1.000000
Latitude    -0.079809  0.011173  0.106389   0.069721   -0.108785  0.002366
Longitude   -0.015176 -0.108197 -0.027540   0.013344    0.099773  0.002476
MedHouseVal  0.688075  0.105623  0.151948  -0.046701   -0.024650 -0.023737

             Latitude  Longitude  MedHouseVal
MedInc      -0.079809  -0.015176     0.688075
HouseAge     0.011173  -0.108197     0.105623
AveRooms     0.106389  -0.027540     0.151948
AveBedrms    0.069721   0.013344    -0.046701
Population  -0.108785   0.099773    -0.024650
AveOccup     0.002366   0.002476    -0.023737
Latitude     1.000000  -0.924664    -0.144160
Longitude   -0.924664   1.000000    -0.045967
MedHouseVal -0.144160  -0.045967     1.000000
```

[5]:
```
plt.figure(figsize=(10, 8))
sns.heatmap(correlation_matrix, annot=True, cmap='coolwarm',fmt='.2f',␣
 ↪linewidths=0.5)
plt.title('Correlational Matrix of California Housing Features')
plt.show()
```



[7]:
```
sns.pairplot(data, diag_kind='kde', plot_kws={'alpha':0.5})
plt.suptitle('Pair Plot of california Housing Features',y=1.02)
plt.show()
```


Pair Plot of california Housing Features
MedInc
HouseAge
AveRooms
AveBedrms
Population
AveOccup
Latitude
Longitude
MedHouseVal

[ ]:

```

```

PRGRM3

February 18, 2026

[2]:
```
import numpy as np
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.decomposition import PCA
import matplotlib.pyplot as plt
```

[4]:
```
iris = load_iris()
data = iris.data
labels = iris.target
label_names = iris.target_names
print(iris)
print(data)
print(labels)
print(label_names)
```

```
{'data': array([[5.1, 3.5, 1.4, 0.2],
       [4.9, 3. , 1.4, 0.2],
       [4.7, 3.2, 1.3, 0.2],
       [4.6, 3.1, 1.5, 0.2],
       [5. , 3.6, 1.4, 0.2],
       [5.4, 3.9, 1.7, 0.4],
       [4.6, 3.4, 1.4, 0.3],
       [5. , 3.4, 1.5, 0.2],
       [4.4, 2.9, 1.4, 0.2],
       [4.9, 3.1, 1.5, 0.1],
       [5.4, 3.7, 1.5, 0.2],
       [4.8, 3.4, 1.6, 0.2],
       [4.8, 3. , 1.4, 0.1],
       [4.3, 3. , 1.1, 0.1],
       [5.8, 4. , 1.2, 0.2],
       [5.7, 4.4, 1.5, 0.4],
       [5.4, 3.9, 1.3, 0.4],
       [5.1, 3.5, 1.4, 0.3],
       [5.7, 3.8, 1.7, 0.3],
       [5.1, 3.8, 1.5, 0.3],
       [5.4, 3.4, 1.7, 0.2],
       [5.1, 3.7, 1.5, 0.4],
       [4.6, 3.6, 1. , 0.2],
```

```
[5.1, 3.3, 1.7, 0.5],
[4.8, 3.4, 1.9, 0.2],
[5. , 3. , 1.6, 0.2],
[5. , 3.4, 1.6, 0.4],
[5.2, 3.5, 1.5, 0.2],
[5.2, 3.4, 1.4, 0.2],
[4.7, 3.2, 1.6, 0.2],
[4.8, 3.1, 1.6, 0.2],
[5.4, 3.4, 1.5, 0.4],
[5.2, 4.1, 1.5, 0.1],
[5.5, 4.2, 1.4, 0.2],
[4.9, 3.1, 1.5, 0.2],
[5. , 3.2, 1.2, 0.2],
[5.5, 3.5, 1.3, 0.2],
[4.9, 3.6, 1.4, 0.1],
[4.4, 3. , 1.3, 0.2],
[5.1, 3.4, 1.5, 0.2],
[5. , 3.5, 1.3, 0.3],
[4.5, 2.3, 1.3, 0.3],
[4.4, 3.2, 1.3, 0.2],
[5. , 3.5, 1.6, 0.6],
[5.1, 3.8, 1.9, 0.4],
[4.8, 3. , 1.4, 0.3],
[5.1, 3.8, 1.6, 0.2],
[4.6, 3.2, 1.4, 0.2],
[5.3, 3.7, 1.5, 0.2],
[5. , 3.3, 1.4, 0.2],
[7. , 3.2, 4.7, 1.4],
[6.4, 3.2, 4.5, 1.5],
[6.9, 3.1, 4.9, 1.5],
[5.5, 2.3, 4. , 1.3],
[6.5, 2.8, 4.6, 1.5],
[5.7, 2.8, 4.5, 1.3],
[6.3, 3.3, 4.7, 1.6],
[4.9, 2.4, 3.3, 1. ],
[6.6, 2.9, 4.6, 1.3],
[5.2, 2.7, 3.9, 1.4],
[5. , 2. , 3.5, 1. ],
[5.9, 3. , 4.2, 1.5],
[6. , 2.2, 4. , 1. ],
[6.1, 2.9, 4.7, 1.4],
[5.6, 2.9, 3.6, 1.3],
[6.7, 3.1, 4.4, 1.4],
[5.6, 3. , 4.5, 1.5],
[5.8, 2.7, 4.1, 1. ],
[6.2, 2.2, 4.5, 1.5],
[5.6, 2.5, 3.9, 1.1],
[5.9, 3.2, 4.8, 1.8],
```

```
[6.1, 2.8, 4. , 1.3],
[6.3, 2.5, 4.9, 1.5],
[6.1, 2.8, 4.7, 1.2],
[6.4, 2.9, 4.3, 1.3],
[6.6, 3. , 4.4, 1.4],
[6.8, 2.8, 4.8, 1.4],
[6.7, 3. , 5. , 1.7],
[6. , 2.9, 4.5, 1.5],
[5.7, 2.6, 3.5, 1. ],
[5.5, 2.4, 3.8, 1.1],
[5.5, 2.4, 3.7, 1. ],
[5.8, 2.7, 3.9, 1.2],
[6. , 2.7, 5.1, 1.6],
[5.4, 3. , 4.5, 1.5],
[6. , 3.4, 4.5, 1.6],
[6.7, 3.1, 4.7, 1.5],
[6.3, 2.3, 4.4, 1.3],
[5.6, 3. , 4.1, 1.3],
[5.5, 2.5, 4. , 1.3],
[5.5, 2.6, 4.4, 1.2],
[6.1, 3. , 4.6, 1.4],
[5.8, 2.6, 4. , 1.2],
[5. , 2.3, 3.3, 1. ],
[5.6, 2.7, 4.2, 1.3],
[5.7, 3. , 4.2, 1.2],
[5.7, 2.9, 4.2, 1.3],
[6.2, 2.9, 4.3, 1.3],
[5.1, 2.5, 3. , 1.1],
[5.7, 2.8, 4.1, 1.3],
[6.3, 3.3, 6. , 2.5],
[5.8, 2.7, 5.1, 1.9],
[7.1, 3. , 5.9, 2.1],
[6.3, 2.9, 5.6, 1.8],
[6.5, 3. , 5.8, 2.2],
[7.6, 3. , 6.6, 2.1],
[4.9, 2.5, 4.5, 1.7],
[7.3, 2.9, 6.3, 1.8],
[6.7, 2.5, 5.8, 1.8],
[7.2, 3.6, 6.1, 2.5],
[6.5, 3.2, 5.1, 2. ],
[6.4, 2.7, 5.3, 1.9],
[6.8, 3. , 5.5, 2.1],
[5.7, 2.5, 5. , 2. ],
[5.8, 2.8, 5.1, 2.4],
[6.4, 3.2, 5.3, 2.3],
[6.5, 3. , 5.5, 1.8],
[7.7, 3.8, 6.7, 2.2],
[7.7, 2.6, 6.9, 2.3],
```

```
       [6. , 2.2, 5. , 1.5],
       [6.9, 3.2, 5.7, 2.3],
       [5.6, 2.8, 4.9, 2. ],
       [7.7, 2.8, 6.7, 2. ],
       [6.3, 2.7, 4.9, 1.8],
       [6.7, 3.3, 5.7, 2.1],
       [7.2, 3.2, 6. , 1.8],
       [6.2, 2.8, 4.8, 1.8],
       [6.1, 3. , 4.9, 1.8],
       [6.4, 2.8, 5.6, 2.1],
       [7.2, 3. , 5.8, 1.6],
       [7.4, 2.8, 6.1, 1.9],
       [7.9, 3.8, 6.4, 2. ],
       [6.4, 2.8, 5.6, 2.2],
       [6.3, 2.8, 5.1, 1.5],
       [6.1, 2.6, 5.6, 1.4],
       [7.7, 3. , 6.1, 2.3],
       [6.3, 3.4, 5.6, 2.4],
       [6.4, 3.1, 5.5, 1.8],
       [6. , 3. , 4.8, 1.8],
       [6.9, 3.1, 5.4, 2.1],
       [6.7, 3.1, 5.6, 2.4],
       [6.9, 3.1, 5.1, 2.3],
       [5.8, 2.7, 5.1, 1.9],
       [6.8, 3.2, 5.9, 2.3],
       [6.7, 3.3, 5.7, 2.5],
       [6.7, 3. , 5.2, 2.3],
       [6.3, 2.5, 5. , 1.9],
       [6.5, 3. , 5.2, 2. ],
       [6.2, 3.4, 5.4, 2.3],
       [5.9, 3. , 5.1, 1.8]]), 'target': array([0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0,
       0, 0, 0, 0, 0, 0, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1,
       1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 1, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2,
       2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2, 2]), 'frame': None,
'target_names': array(['setosa', 'versicolor', 'virginica'], dtype='<U10'),
'DESCR': '.. _iris_dataset:\n\nIris plants
dataset\n--------------------\n\n**Data Set Characteristics:**\n\n:Number of
Instances: 150 (50 in each of three classes)\n:Number of Attributes: 4 numeric,
predictive attributes and the class\n:Attribute Information:\n    - sepal length
in cm\n    - sepal width in cm\n    - petal length in cm\n    - petal width in
cm\n    - class:\n            - Iris-Setosa\n            - Iris-Versicolour\n
- Iris-Virginica\n\n:Summary Statistics:\n\n============== ==== ==== =======
===== ====================\n                Min  Max   Mean    SD   Class
Correlation\n============== ==== ==== ======= ===== ====================\nsepal
```

length: 4.3 7.9 5.84 0.83 0.7826\nsepal width: 2.0 4.4 3.05 0.43 -0.4194\npetal length: 1.0 6.9 3.76 1.76 0.9490 (high!)\npetal width: 0.1 2.5 1.20 0.76 0.9565 (high!)\n============== ==== ==== ======= ===== ====================\n\n:Missing Attribute Values: None\n:Class Distribution: 33.3% for each of 3 classes.\n:Creator: R.A. Fisher\n:Donor: Michael Marshall (MARSHALL%PLU@io.arc.nasa.gov)\n:Date: July, 1988\n\nThe famous Iris database, first used by Sir R.A. Fisher. The dataset is taken\nfrom Fisher\'s paper. Note that it\'s the same as in R, but not as in the UCI\nMachine Learning Repository, which has two wrong data points.\n\nThis is perhaps the best known database to be found in the\npattern recognition literature. Fisher\'s paper is a classic in the field and\nis referenced frequently to this day. (See Duda & Hart, for example.) The\ndata set contains 3 classes of 50 instances each, where each class refers to a\ntype of iris plant. One class is linearly separable from the other 2; the\nlatter are NOT linearly separable from each other.\n\n.. dropdown:: References\n\n - Fisher, R.A. "The use of multiple measurements in taxonomic problems"\n Annual Eugenics, 7, Part II, 179-188 (1936); also in "Contributions to\n Mathematical Statistics" (John Wiley, NY, 1950).\n - Duda, R.O., & Hart, P.E. (1973) Pattern Classification and Scene Analysis.\n (Q327.D83) John Wiley & Sons. ISBN 0-471-22361-1. See page 218.\n - Dasarathy, B.V. (1980) "Nosing Around the Neighborhood: A New System\n Structure and Classification Rule for Recognition in Partially Exposed\n Environments". IEEE Transactions on Pattern Analysis and Machine\n Intelligence, Vol. PAMI-2, No. 1, 67-71.\n - Gates, G.W. (1972) "The Reduced Nearest Neighbor Rule". IEEE Transactions\n on Information Theory, May 1972, 431-433.\n - See also: 1988 MLC Proceedings, 54-64. Cheeseman et al"s AUTOCLASS II\n conceptual clustering system finds 3 classes in the data.\n - Many, many more …\n', 'feature_names': ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)'], 'filename': 'iris.csv', 'data_module': 'sklearn.datasets.data'}

```
[[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]
 [5.4 3.9 1.7 0.4]
 [4.6 3.4 1.4 0.3]
 [5.  3.4 1.5 0.2]
 [4.4 2.9 1.4 0.2]
 [4.9 3.1 1.5 0.1]
 [5.4 3.7 1.5 0.2]
 [4.8 3.4 1.6 0.2]
 [4.8 3.  1.4 0.1]
 [4.3 3.  1.1 0.1]
 [5.8 4.  1.2 0.2]
 [5.7 4.4 1.5 0.4]
 [5.4 3.9 1.3 0.4]
 [5.1 3.5 1.4 0.3]
 [5.7 3.8 1.7 0.3]
```

```
[5.1 3.8 1.5 0.3]
[5.4 3.4 1.7 0.2]
[5.1 3.7 1.5 0.4]
[4.6 3.6 1.  0.2]
[5.1 3.3 1.7 0.5]
[4.8 3.4 1.9 0.2]
[5.  3.  1.6 0.2]
[5.  3.4 1.6 0.4]
[5.2 3.5 1.5 0.2]
[5.2 3.4 1.4 0.2]
[4.7 3.2 1.6 0.2]
[4.8 3.1 1.6 0.2]
[5.4 3.4 1.5 0.4]
[5.2 4.1 1.5 0.1]
[5.5 4.2 1.4 0.2]
[4.9 3.1 1.5 0.2]
[5.  3.2 1.2 0.2]
[5.5 3.5 1.3 0.2]
[4.9 3.6 1.4 0.1]
[4.4 3.  1.3 0.2]
[5.1 3.4 1.5 0.2]
[5.  3.5 1.3 0.3]
[4.5 2.3 1.3 0.3]
[4.4 3.2 1.3 0.2]
[5.  3.5 1.6 0.6]
[5.1 3.8 1.9 0.4]
[4.8 3.  1.4 0.3]
[5.1 3.8 1.6 0.2]
[4.6 3.2 1.4 0.2]
[5.3 3.7 1.5 0.2]
[5.  3.3 1.4 0.2]
[7.  3.2 4.7 1.4]
[6.4 3.2 4.5 1.5]
[6.9 3.1 4.9 1.5]
[5.5 2.3 4.  1.3]
[6.5 2.8 4.6 1.5]
[5.7 2.8 4.5 1.3]
[6.3 3.3 4.7 1.6]
[4.9 2.4 3.3 1. ]
[6.6 2.9 4.6 1.3]
[5.2 2.7 3.9 1.4]
[5.  2.  3.5 1. ]
[5.9 3.  4.2 1.5]
[6.  2.2 4.  1. ]
[6.1 2.9 4.7 1.4]
[5.6 2.9 3.6 1.3]
[6.7 3.1 4.4 1.4]
[5.6 3.  4.5 1.5]
```

```
[5.8 2.7 4.1 1. ]
[6.2 2.2 4.5 1.5]
[5.6 2.5 3.9 1.1]
[5.9 3.2 4.8 1.8]
[6.1 2.8 4.  1.3]
[6.3 2.5 4.9 1.5]
[6.1 2.8 4.7 1.2]
[6.4 2.9 4.3 1.3]
[6.6 3.  4.4 1.4]
[6.8 2.8 4.8 1.4]
[6.7 3.  5.  1.7]
[6.  2.9 4.5 1.5]
[5.7 2.6 3.5 1. ]
[5.5 2.4 3.8 1.1]
[5.5 2.4 3.7 1. ]
[5.8 2.7 3.9 1.2]
[6.  2.7 5.1 1.6]
[5.4 3.  4.5 1.5]
[6.  3.4 4.5 1.6]
[6.7 3.1 4.7 1.5]
[6.3 2.3 4.4 1.3]
[5.6 3.  4.1 1.3]
[5.5 2.5 4.  1.3]
[5.5 2.6 4.4 1.2]
[6.1 3.  4.6 1.4]
[5.8 2.6 4.  1.2]
[5.  2.3 3.3 1. ]
[5.6 2.7 4.2 1.3]
[5.7 3.  4.2 1.2]
[5.7 2.9 4.2 1.3]
[6.2 2.9 4.3 1.3]
[5.1 2.5 3.  1.1]
[5.7 2.8 4.1 1.3]
[6.3 3.3 6.  2.5]
[5.8 2.7 5.1 1.9]
[7.1 3.  5.9 2.1]
[6.3 2.9 5.6 1.8]
[6.5 3.  5.8 2.2]
[7.6 3.  6.6 2.1]
[4.9 2.5 4.5 1.7]
[7.3 2.9 6.3 1.8]
[6.7 2.5 5.8 1.8]
[7.2 3.6 6.1 2.5]
[6.5 3.2 5.1 2. ]
[6.4 2.7 5.3 1.9]
[6.8 3.  5.5 2.1]
[5.7 2.5 5.  2. ]
[5.8 2.8 5.1 2.4]
```

```
 [6.4 3.2 5.3 2.3]
 [6.5 3.  5.5 1.8]
 [7.7 3.8 6.7 2.2]
 [7.7 2.6 6.9 2.3]
 [6.  2.2 5.  1.5]
 [6.9 3.2 5.7 2.3]
 [5.6 2.8 4.9 2. ]
 [7.7 2.8 6.7 2. ]
 [6.3 2.7 4.9 1.8]
 [6.7 3.3 5.7 2.1]
 [7.2 3.2 6.  1.8]
 [6.2 2.8 4.8 1.8]
 [6.1 3.  4.9 1.8]
 [6.4 2.8 5.6 2.1]
 [7.2 3.  5.8 1.6]
 [7.4 2.8 6.1 1.9]
 [7.9 3.8 6.4 2. ]
 [6.4 2.8 5.6 2.2]
 [6.3 2.8 5.1 1.5]
 [6.1 2.6 5.6 1.4]
 [7.7 3.  6.1 2.3]
 [6.3 3.4 5.6 2.4]
 [6.4 3.1 5.5 1.8]
 [6.  3.  4.8 1.8]
 [6.9 3.1 5.4 2.1]
 [6.7 3.1 5.6 2.4]
 [6.9 3.1 5.1 2.3]
 [5.8 2.7 5.1 1.9]
 [6.8 3.2 5.9 2.3]
 [6.7 3.3 5.7 2.5]
 [6.7 3.  5.2 2.3]
 [6.3 2.5 5.  1.9]
 [6.5 3.  5.2 2. ]
 [6.2 3.4 5.4 2.3]
 [5.9 3.  5.1 1.8]]
[0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0 0
 0 0 0 0 0 0 0 0 0 0 0 0 0 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1
 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 1 2 2 2 2 2 2 2 2 2 2 2
 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2 2
 2 2]
['setosa' 'versicolor' 'virginica']
```

[7]:
```python
iris_df = pd.DataFrame(data, columns=iris.feature_names)
print(iris_df)
```

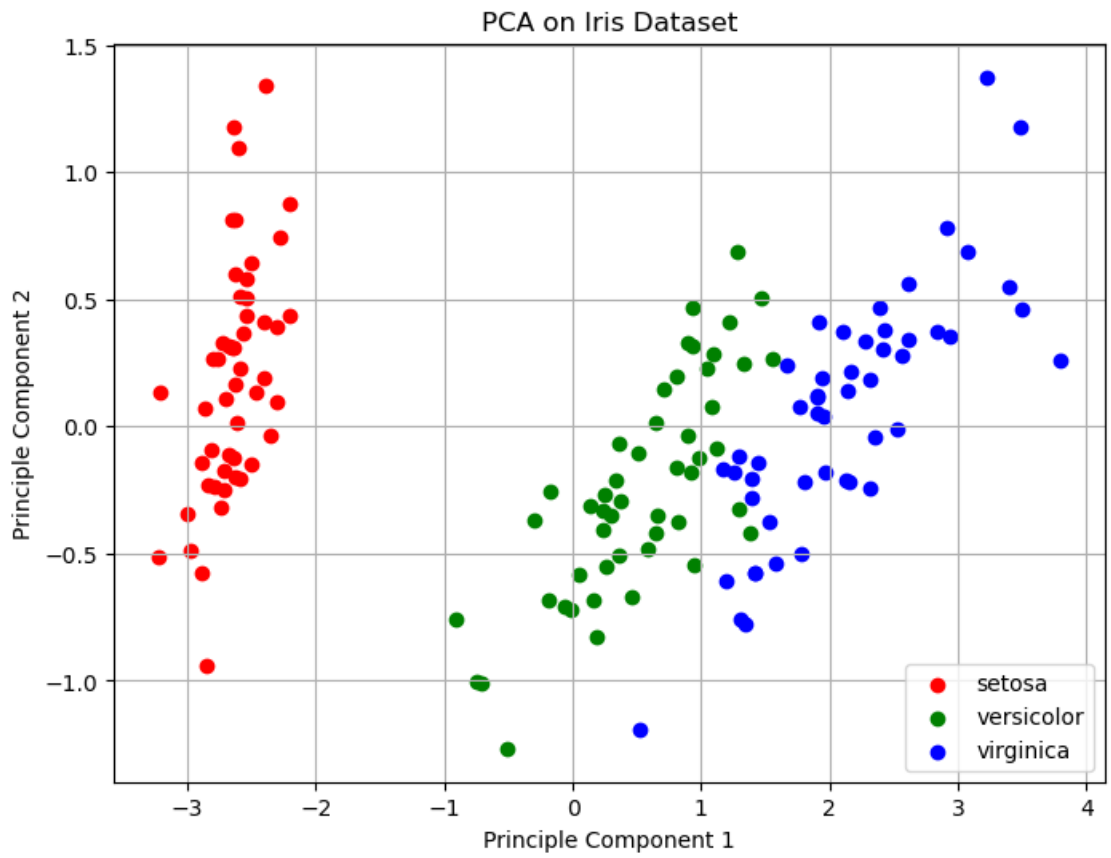
```
     sepal length (cm)  sepal width (cm)  petal length (cm)  petal width (cm)
0                  5.1               3.5                1.4               0.2
1                  4.9               3.0                1.4               0.2
```

```
2                  4.7                3.2                1.3                0.2
3                  4.6                3.1                1.5                0.2
4                  5.0                3.6                1.4                0.2
..                 …                  …                  …                  …
145                6.7                3.0                5.2                2.3
146                6.3                2.5                5.0                1.9
147                6.5                3.0                5.2                2.0
148                6.2                3.4                5.4                2.3
149                5.9                3.0                5.1                1.8

[150 rows x 4 columns]
```

[9]:
```
pca = PCA (n_components=2)
data_reduced = pca.fit_transform(data)
```

[10]:
```
reduced_df = pd.DataFrame(data_reduced, columns=['Principle Component 1',␣
 ↪'Principle Component 2'])
reduced_df ['label'] = labels
```

[15]:
```
plt.figure(figsize=(8,6))
colors = ['r' ,'g', 'b']
for i, label in enumerate(np.unique(labels)):
    plt.scatter(
        reduced_df[reduced_df['label'] == label]['Principle Component 1'],
        reduced_df[reduced_df['label'] == label]['Principle Component 2'],
        label=label_names[label],
        color=colors[i])
plt.title('PCA on Iris Dataset')
plt.xlabel('Principle Component 1')
plt.ylabel('Principle Component 2')
plt.legend()
plt.grid()
plt.show()
```


PCA on Iris Dataset
Principle Component 2
Principle Component 1
1.5
1.0
0.5
0.0
−0.5
−1.0
−3
−2
−1
0
1
2
3
4
setosa
versicolor
virginica

```
[ ]:
```

PRGRM4

March 11, 2026

[6]:
```python
import pandas as pd
def find_s_algorithm(file_path):
    data = pd.read_csv(file_path)
    print("Training Data.")
    print(data)
    attributes = data.columns[:-1]
    class_label = data.columns[-1]

    hypothesis = ['?' for _ in attributes]
    for index,row in data.iterrows():
        if row[class_label] == 'Yes':
            for i,value in enumerate(row[attributes]):
                if hypothesis[i] == '?' or hypothesis[i] == value:
                    hypothesis[i] = value
                else:
                    hypothesis[i] = '?'
    return hypothesis
file_path ='training_data.csv'
hypothesis = find_s_algorithm(file_path)
print("\n The Final hypothesis is :",hypothesis)
```

```
Training Data.
   Sunny  Warm  Normal  Strong Warm.1    Same  Yes
0  Sunny  Warm    High  Strong   Warm    Same  Yes
1  Rainy  Cold    High  Strong   Warm  Change   No
2  Sunny  Warm    High  Strong   Cool  Change  Yes
3  Rainy  Cold  Normal  Strong   Cool    Same   No

 The Final hypothesis is : ['Sunny', 'Warm', 'High', 'Strong', '?', '?']
```

[ ]:
```python

```

PRGRM5

March 11, 2026

[ ]:
```
import numpy as np
from collections import Counter
```

[ ]:
```
np.random.seed(42)
x_values = np.random.rand(100)

x_train =x_values[:50]
x_test = x_values[50:]
y_train =[]

for x in x_train:
    if x <=0.5:
        y_train.append("Class1")
    else:
        y_train.append("Class2")
def knn_classify(x_train,y_train,x_test,k):
    predictions = []
    for test_point in x_test:
        distances=[]
        for i in range(len(x_train)):
            distance = abs(test_point - x_train[i])
            distances.append((distance,y_train[i]))
        distances.sort(key = lambda x:x[0])
        k_neighbors = distances[:k]
        classes = [neighbor[1] for neighbor in k_neighbors]
        prediction = Counter(classes).most_common(1)[0][0]
        predictions.append(prediction)
    return predictions
k_values = [1,2,3,4,5,20,30]
for k in k_values:
    predictions = knn_classify(x_train,y_train,x_test,k)
    print(f"\nResults for k={k}")
    for i in range(len(predictions)):
        print(f"x={x_test[i]:.4f}--> {predictions[i]}")
```

```
Results for k=1
```

```
x=0.9696--> Class2
x=0.7751--> Class2
x=0.9395--> Class2
x=0.8948--> Class2
x=0.5979--> Class2
x=0.9219--> Class2
x=0.0885--> Class1
x=0.1960--> Class1
x=0.0452--> Class1
x=0.3253--> Class1
x=0.3887--> Class1
x=0.2713--> Class1
x=0.8287--> Class2
x=0.3568--> Class1
x=0.2809--> Class1
x=0.5427--> Class2
x=0.1409--> Class1
x=0.8022--> Class2
x=0.0746--> Class1
x=0.9869--> Class2
x=0.7722--> Class2
x=0.1987--> Class1
x=0.0055--> Class1
x=0.8155--> Class2
x=0.7069--> Class2
x=0.7290--> Class2
x=0.7713--> Class2
x=0.0740--> Class1
x=0.3585--> Class1
x=0.1159--> Class1
x=0.8631--> Class2
x=0.6233--> Class2
x=0.3309--> Class1
x=0.0636--> Class1
x=0.3110--> Class1
x=0.3252--> Class1
x=0.7296--> Class2
x=0.6376--> Class2
x=0.8872--> Class2
x=0.4722--> Class1
x=0.1196--> Class1
x=0.7132--> Class2
x=0.7608--> Class2
x=0.5613--> Class2
x=0.7710--> Class2
x=0.4938--> Class1
x=0.5227--> Class2
x=0.4275--> Class1
```

```
x=0.0254--> Class1
x=0.1079--> Class1

Results for k=2
x=0.9696--> Class2
x=0.7751--> Class2
x=0.9395--> Class2
x=0.8948--> Class2
x=0.5979--> Class2
x=0.9219--> Class2
x=0.0885--> Class1
x=0.1960--> Class1
x=0.0452--> Class1
x=0.3253--> Class1
x=0.3887--> Class1
x=0.2713--> Class1
x=0.8287--> Class2
x=0.3568--> Class1
x=0.2809--> Class1
x=0.5427--> Class2
x=0.1409--> Class1
x=0.8022--> Class2
x=0.0746--> Class1
x=0.9869--> Class2
x=0.7722--> Class2
x=0.1987--> Class1
x=0.0055--> Class1
x=0.8155--> Class2
x=0.7069--> Class2
x=0.7290--> Class2
x=0.7713--> Class2
x=0.0740--> Class1
x=0.3585--> Class1
x=0.1159--> Class1
x=0.8631--> Class2
x=0.6233--> Class2
x=0.3309--> Class1
x=0.0636--> Class1
x=0.3110--> Class1
x=0.3252--> Class1
x=0.7296--> Class2
x=0.6376--> Class2
x=0.8872--> Class2
x=0.4722--> Class1
x=0.1196--> Class1
x=0.7132--> Class2
x=0.7608--> Class2
x=0.5613--> Class2
```

```
x=0.7710--> Class2
x=0.4938--> Class1
x=0.5227--> Class2
x=0.4275--> Class1
x=0.0254--> Class1
x=0.1079--> Class1

Results for k=3
x=0.9696--> Class2
x=0.7751--> Class2
x=0.9395--> Class2
x=0.8948--> Class2
x=0.5979--> Class2
x=0.9219--> Class2
x=0.0885--> Class1
x=0.1960--> Class1
x=0.0452--> Class1
x=0.3253--> Class1
x=0.3887--> Class1
x=0.2713--> Class1
x=0.8287--> Class2
x=0.3568--> Class1
x=0.2809--> Class1
x=0.5427--> Class2
x=0.1409--> Class1
x=0.8022--> Class2
x=0.0746--> Class1
x=0.9869--> Class2
x=0.7722--> Class2
x=0.1987--> Class1
x=0.0055--> Class1
x=0.8155--> Class2
x=0.7069--> Class2
x=0.7290--> Class2
x=0.7713--> Class2
x=0.0740--> Class1
x=0.3585--> Class1
x=0.1159--> Class1
x=0.8631--> Class2
x=0.6233--> Class2
x=0.3309--> Class1
x=0.0636--> Class1
x=0.3110--> Class1
x=0.3252--> Class1
x=0.7296--> Class2
x=0.6376--> Class2
x=0.8872--> Class2
x=0.4722--> Class1
```

```
x=0.1196--> Class1
x=0.7132--> Class2
x=0.7608--> Class2
x=0.5613--> Class2
x=0.7710--> Class2
x=0.4938--> Class2
x=0.5227--> Class2
x=0.4275--> Class1
x=0.0254--> Class1
x=0.1079--> Class1

Results for k=4
x=0.9696--> Class2
x=0.7751--> Class2
x=0.9395--> Class2
x=0.8948--> Class2
x=0.5979--> Class2
x=0.9219--> Class2
x=0.0885--> Class1
x=0.1960--> Class1
x=0.0452--> Class1
x=0.3253--> Class1
x=0.3887--> Class1
x=0.2713--> Class1
x=0.8287--> Class2
x=0.3568--> Class1
x=0.2809--> Class1
x=0.5427--> Class2
x=0.1409--> Class1
x=0.8022--> Class2
x=0.0746--> Class1
x=0.9869--> Class2
x=0.7722--> Class2
x=0.1987--> Class1
x=0.0055--> Class1
x=0.8155--> Class2
x=0.7069--> Class2
x=0.7290--> Class2
x=0.7713--> Class2
x=0.0740--> Class1
x=0.3585--> Class1
x=0.1159--> Class1
x=0.8631--> Class2
x=0.6233--> Class2
x=0.3309--> Class1
x=0.0636--> Class1
x=0.3110--> Class1
x=0.3252--> Class1
```

```
x=0.7296--> Class2
x=0.6376--> Class2
x=0.8872--> Class2
x=0.4722--> Class1
x=0.1196--> Class1
x=0.7132--> Class2
x=0.7608--> Class2
x=0.5613--> Class2
x=0.7710--> Class2
x=0.4938--> Class2
x=0.5227--> Class2
x=0.4275--> Class1
x=0.0254--> Class1
x=0.1079--> Class1

Results for k=5
x=0.9696--> Class2
x=0.7751--> Class2
x=0.9395--> Class2
x=0.8948--> Class2
x=0.5979--> Class2
x=0.9219--> Class2
x=0.0885--> Class1
x=0.1960--> Class1
x=0.0452--> Class1
x=0.3253--> Class1
x=0.3887--> Class1
x=0.2713--> Class1
x=0.8287--> Class2
x=0.3568--> Class1
x=0.2809--> Class1
x=0.5427--> Class2
x=0.1409--> Class1
x=0.8022--> Class2
x=0.0746--> Class1
x=0.9869--> Class2
x=0.7722--> Class2
x=0.1987--> Class1
x=0.0055--> Class1
x=0.8155--> Class2
x=0.7069--> Class2
x=0.7290--> Class2
x=0.7713--> Class2
x=0.0740--> Class1
x=0.3585--> Class1
x=0.1159--> Class1
x=0.8631--> Class2
x=0.6233--> Class2
```

```
x=0.3309--> Class1
x=0.0636--> Class1
x=0.3110--> Class1
x=0.3252--> Class1
x=0.7296--> Class2
x=0.6376--> Class2
x=0.8872--> Class2
x=0.4722--> Class1
x=0.1196--> Class1
x=0.7132--> Class2
x=0.7608--> Class2
x=0.5613--> Class2
x=0.7710--> Class2
x=0.4938--> Class2
x=0.5227--> Class2
x=0.4275--> Class1
x=0.0254--> Class1
x=0.1079--> Class1

Results for k=20
x=0.9696--> Class2
x=0.7751--> Class2
x=0.9395--> Class2
x=0.8948--> Class2
x=0.5979--> Class2
x=0.9219--> Class2
x=0.0885--> Class1
x=0.1960--> Class1
x=0.0452--> Class1
x=0.3253--> Class1
x=0.3887--> Class1
x=0.2713--> Class1
x=0.8287--> Class2
x=0.3568--> Class1
x=0.2809--> Class1
x=0.5427--> Class2
x=0.1409--> Class1
x=0.8022--> Class2
x=0.0746--> Class1
x=0.9869--> Class2
x=0.7722--> Class2
x=0.1987--> Class1
x=0.0055--> Class1
x=0.8155--> Class2
x=0.7069--> Class2
x=0.7290--> Class2
x=0.7713--> Class2
x=0.0740--> Class1
```

```
x=0.3585--> Class1
x=0.1159--> Class1
x=0.8631--> Class2
x=0.6233--> Class2
x=0.3309--> Class1
x=0.0636--> Class1
x=0.3110--> Class1
x=0.3252--> Class1
x=0.7296--> Class2
x=0.6376--> Class2
x=0.8872--> Class2
x=0.4722--> Class1
x=0.1196--> Class1
x=0.7132--> Class2
x=0.7608--> Class2
x=0.5613--> Class2
x=0.7710--> Class2
x=0.4938--> Class2
x=0.5227--> Class2
x=0.4275--> Class1
x=0.0254--> Class1
x=0.1079--> Class1

Results for k=30
x=0.9696--> Class2
x=0.7751--> Class2
x=0.9395--> Class2
x=0.8948--> Class2
x=0.5979--> Class2
x=0.9219--> Class2
x=0.0885--> Class1
x=0.1960--> Class1
x=0.0452--> Class1
x=0.3253--> Class1
x=0.3887--> Class1
x=0.2713--> Class1
x=0.8287--> Class2
x=0.3568--> Class1
x=0.2809--> Class1
x=0.5427--> Class2
x=0.1409--> Class1
x=0.8022--> Class2
x=0.0746--> Class1
x=0.9869--> Class2
x=0.7722--> Class2
x=0.1987--> Class1
x=0.0055--> Class1
x=0.8155--> Class2
```

```
x=0.7069--> Class2
x=0.7290--> Class2
x=0.7713--> Class2
x=0.0740--> Class1
x=0.3585--> Class1
x=0.1159--> Class1
x=0.8631--> Class2
x=0.6233--> Class2
x=0.3309--> Class1
x=0.0636--> Class1
x=0.3110--> Class1
x=0.3252--> Class1
x=0.7296--> Class2
x=0.6376--> Class2
x=0.8872--> Class2
x=0.4722--> Class1
x=0.1196--> Class1
x=0.7132--> Class2
x=0.7608--> Class2
x=0.5613--> Class2
x=0.7710--> Class2
x=0.4938--> Class1
x=0.5227--> Class2
x=0.4275--> Class1
x=0.0254--> Class1
x=0.1079--> Class1
```

[ ]:

```
```

PRGRM6

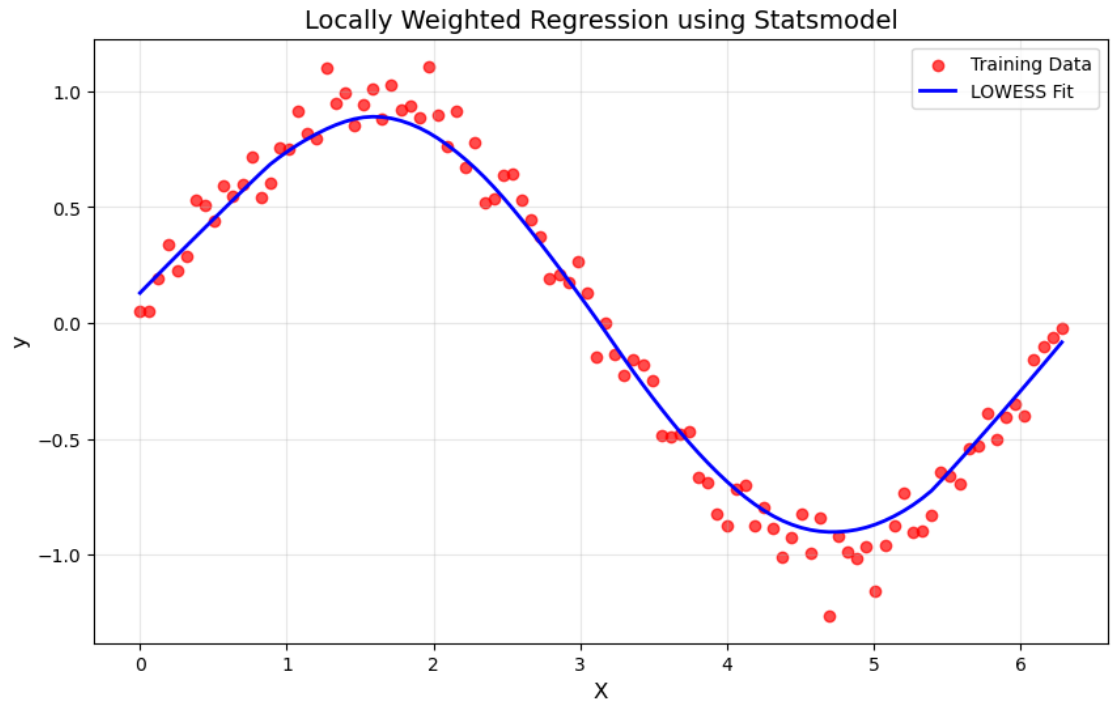
March 11, 2026

[1]:
```
import numpy as np
import matplotlib.pyplot as plt
from statsmodels.nonparametric.smoothers_lowess import lowess
```

[4]:
```
np.random.seed(42)
X = np.linspace(0,2 * np.pi,100)
y = np.sin(X)+0.1 * np.random.randn(100)
```

[5]:
```
lowess_result = lowess(y,X,frac=0.3)
```

[7]:
```
plt.figure(figsize=(10, 6))
plt.scatter(X,y,color='red', label='Training Data', alpha=0.7)
plt.plot(lowess_result[:, 0], lowess_result[:,1 ], color='blue', label='LOWESS␣
↪Fit', linewidth=2)
plt.xlabel('X',fontsize=12)
plt.ylabel('y',fontsize=12)
plt.title('Locally Weighted Regression using Statsmodel', fontsize=14)
plt.legend(fontsize=10)
plt.grid(alpha=0.3)
plt.show()
```


Locally Weighted Regression using Statsmodel
Training Data
LOWESS Fit
1.0
0.5
0.0
−0.5
−1.0
y
0
1
2
3
4
5
6
X

```
[ ]:
```